CSC3150 OS Midterm · Side A：概念 / 系统编程 / 文件与 IPC / 同步（L1–L9）

答题：结论 → 机制 → 例子/反例

1 · OS 是什么 (L1)

操作系统（OS）：v1=把（多个）应用与（多样）硬件资源连接起来的那层软件；v2=为应用实现一台“虚拟机”，其接口比裸硬件更方便（可移植、可靠、安全）。三顶帽子：**Referee（裁判）**=管理资源的保护、隔离与共享（进程=运行中程序的实例，§2；双模式，§2；调度=决定谁何时用 CPU，§14；pipe/socket=进程间通信通道，§7）；**Illusionist（魔术师）**=为物理资源提供干净易用的抽象，用**虚拟化**（让每个程序以为独占整台机器、资源无限）掩盖硬件限制（独占、无限资源、不存在的硬件能力；cpu.c 三进程交替输出 A/B/C=**分时**（多个程序轮流使用 CPU）+定时器**抢占**（OS 借定时器中断强行收回 CPU）；memory.c 两进程打印同一虚址 0x200000 但各自 *p 独立递增=独立地址空间）；**Glue（胶水）**=提供一组公共服务（文件系统、网络、GUI 等），便于共享与复用。Win3.1/MacOS9 无抢占：一个死循环卡死全机。Referee 三职责：故障隔离（进程、双模式）、资源共享（调度：选谁跑、怎么分物理资源）、通信（pipe/socket）。Illusionist 三幻觉：**All alone**（独占）、**All powerful**（资源无限）、**All expressive**（提供硬件没有的能力）。Glue：共享更容易（大家用同一套原语）、最大化复用、组件可独立演进。memory.c 两进程题答案：(b) 各自 1,1,2,2,3,3（不是共享计数）。**评价指标**：**Overhead**=实现某抽象额外付出的资源代价；**Fairness**=资源在应用间分配是否公平；**Portability**=硬件变化时应用要改多少（可移植的抽象不随硬件变；靠 **HAL** 硬件抽象层与抽象机接口）；**Reliability**=系统是否做它该做的事（**Availability** 可用性由 MTTF 平均无故障时间与 MTTR 平均修复时间决定：$A=\frac{\mathrm{MTTF}}{\mathrm{MTTF+MTTR}}$）；**Security**=受攻击时仍保持预期功能（**Integrity**=系统运行不能被恶意攻击者破坏；**Privacy**=数据只对授权用户可访问）；**Performance**=在 **Response time**（完成一个任务要多久）、**Throughput**（单位时间完成多少任务）、**Predictability**（性能指标是否随时间稳定）上满足用户与管理员预期。**为何难**：设备种类多（Bell 定律：每 10 年一个新设备类）、时间尺度跨 8 个数量级（L1 0.5ns · mutex 25ns · 内存 100ns · SSD 随机读 ≈100µs · 磁盘寻道 10ms · 跨洲 RTT（往返时延）150ms）、安全关键。**复杂度为何暴涨**：硬件更智能、更高可靠/安全、更高性能（更高效/更并行的代码）、能效、历史遗留(legacy)。教材：OS Concepts（Silberschatz）、OSTEP、RISC-V Reader；作业基于 MIT **xv6**（教学用类 UNIX 小内核），4 个作业各 12.5%（AI 视为上过本课的同学，禁止抄）。课程：作业 50%、期中 **20%** 笔试、期末 **30%** 上机。

2 · 四大基本概念 (L2)

Thread（线程）=单一、唯一的执行上下文，完整描述程序状态：**PC**（程序计数器，指向下一条指令）、寄存器、执行标志 Flags、栈（**SP** 栈指针=栈顶地址）、内存状态；线程**运行中**=其状态驻留在 CPU 寄存器里；**挂起**=状态未装入 CPU，寄存器拷贝保存在内存的 **TCB**（线程控制块）。**Address space（地址空间）**=程序可访问（读/写）的地址集合及其相关状态，可与物理内存不同（即虚拟地址空间）（32 位 $2^{32}\approx4\times10^{9}$，64 位 $2^{64}\approx1.8\times10^{19}$）；读写某地址可能：普通内存 / 忽略写 / **MMIO**（内存映射 I/O：读写该地址实际是访问设备）/ 异常 / 与他进程通信。布局（低→高）：**code**（机器指令）→ **static data**（全局、static 变量）→ **heap**（malloc 动态分配，↑增长）→ … → **stack**（函数调用时自动分配的局部变量/参数/返回地址，↓增长）。int *p=malloc(4)：p 在栈，*p 在堆。**Process（进程）**=运行中程序的一个实例；=带受限权限的程序执行环境=地址空间+≥1 线程+fd/文件系统上下文等资源；**保护 vs 效率的根本权衡**：进程内通信容易，进程间难。线程=“主动”并发；地址空间=“被动”保护。**Dual mode（双模式）**=硬件至少提供两种运行模式（≥1 个模式位）：**内核态**（kernel/supervisor mode，可访问全部硬件）与**用户态**（user mode，只能有限访问），只有“系统”能访问某些资源；结合**地址翻译**（硬件把程序使用的虚拟地址映射为物理地址）可把程序彼此隔离、把 OS 与程序隔离。用户态禁止：改页表指针/地址转换寄存器、关中断、直接访问硬件(I/O)、写内核内存。转换：syscall / interrupt / exception（进，见下表），rti/RTU（回；rti/rfi=从中断返回的指令）。目的：保护 OS 和其他进程不受用户程序破坏（可靠性/安全/隐私/公平）。**Base & Bound**=每个进程一对基址/界限寄存器，限定它只能访问 [Base, Bound) 内的内存。v1 加载时重定位（需 **relocating loader**：加载时把程序地址改写成物理地址；地址通路无加法，仅比较 Base≤addr<Bound）；v2 运行时硬件转换（物理=Base+虚址，检查 <Bound）。程序不能触碰 OS 和他进程。设置 Base/Bound 是**特权指令**（只能在内核态执行的指令）；RTU=Return To Usermode，返回用户态的特权指令。**用户→内核 3 种转移**，都是 **unprogrammed control transfer**（不由运行中进程的直接指令发起的、切到 OS 的控制转移），都经**中断向量表**（按中断号索引、存各中断处理程序地址与属性的表）跳到 intrpHandler_i，用户不能指定内核地址。**Syscall**=进程请求系统服务（像函数调用，但在进程“之外”）；**Interrupt**=外部异步事件触发切换；**Trap/Exception**=进程内部同步事件触发切换：

类型	同步?	例
Syscall	同步/主动	像 **RPC**（远程过程调用）：syscall 号与参数放寄存器再 trap
Interrupt	异步/外部	定时器、I/O 完成，与当前进程无关
Trap/ Exception	同步/内部	段错误（访问无权限/未映射内存=保护违例）、除 0、非法指令

虚拟化层级：OS 把硬件虚拟给进程；**Hypervisor**=创建虚拟机（VM）的软件，**VM** 把硬件虚拟给 OS、提供相同的硬件抽象（VM 间故障隔离：一个 OS 崩溃不影响其他 OS）。也可以是 OS 0 上跑 hypervisor（**托管式**=hypervisor 作为宿主 OS 上的一个程序运行）。**运行程序的流程**：编辑→编译成 a.out→把指令段和数据段装入内存→建栈和堆→把控制权交给程序→在保护 OS 与程序的同时为其提供服务。**为何一个地址空间放多个线程**：并行（利用多核）+ 并发（方便处理 I/O 与同时发生的事件）。**单核多 vCPU**（vCPU=靠时间复用给每个程序造出的“虚拟 CPU”）：每个 vCPU 需保存 PC/SP/寄存器；切换=存当前状态块、载入新状态块；触发：定时器、主动 **yield**（线程自愿让出 CPU）、I/O 等。**无保护共享模型**（线程可读写彼此数据与指令）：嵌入式、Win3.1/早期 Mac（只靠 yield 切换）、Win95–ME（yield+定时器）。**保护机制**：主=限制程序地址空间到物理内存的映射（只能碰被映射的）；辅=特权指令、I/O 指令、特殊寄存器、syscall 处理、子系统实现（如文件访问权限）。模式转换图：user→kernel：syscall/interrupt/exception；kernel→user：rtn/rfi/exec；exit 退出。

3 · 进程切换、PCB、线程 (L3)

上下文切换（context switch）=保存当前进程/线程的寄存器等状态、装入另一个的状态。**B&B 下切换**（sysmode=模式位，uPC=保存的用户态 PC）：P1 用户态(sysmode=0) → 定时器中断：硬件置 sysmode=1、存 uPC、PC=IntrpVector[i] → 内核存 P1 寄存器/uPC 到 PCB、换内核栈、调度器选 P2、装 P2 的 Base/Bound/寄存器/uPC → RTU。**PCB**（进程控制块）=内核用来表示每个进程的数据结构，含：状态（running/ready/blocked）、寄存器、**PID**（进程标识号）、用户、可执行文件、优先级、执行时间、内存空间/地址转换、fd。**调度器**（scheduler）=决定哪个进程/线程何时、多久获得 CPU 的机制：if(ready) run(select(PCBs)) else run_idle()；选谁=**策略（policy）**。**线程状态**（**admit**=新线程被接纳进就绪队列；**dispatch**=调度器选中一个就绪线程让它运行；**ready 队列**=就绪线程排队等 CPU 的队列）：**New**=正在创建；**Ready**=等待被分配处理器；**Running**=指令正在执行；**Waiting/ Blocked**=等待某事件发生（如 I/O）；**Terminated**=已执行完。New→Ready→Running→(Waiting/Blocked→Ready)→Terminated；Running→Ready（被抢占/yield）。**共享 vs 私有**：共享=内存内容（全局变量、堆）、I/O 状态（fd、网络连接）、代码；私有(TCB)=寄存器(含 PC)、执行栈。每线程有用户栈+内核栈（内核栈及其支撑结构常称 **kernel thread**）。多线程栈布局问题：栈间距、最大尺寸、溢出检测（**guard page**=栈之间不可访问的页，越界即触发异常）。**Linux**：一切皆 task_struct（内核调度的唯一实体）；clone(...)=Linux 创建新 task 的 syscall，标志位决定与父共享哪些资源；clone(CLONE_VM|CLONE_FS|CLONE_FILES|CLONE_SIGHAND) 建线程（共享地址空间/文件系统/打开文件/信号处理）；不带这些标志≈fork。进程=线程的容器。**PCB vs TCB（L7）**：PCB 有代码、数据、堆的指针和 fd 表；每个 TCB 有保存的寄存器、栈和元数据。同进程线程间无隔离（共享地址空间与 I/O 状态 fd），但各有执行上下文（栈、寄存器）。**术语**：**Multiprocessing**=多 CPU（核）；**Multiprogramming**=多个作业/进程；**Multithreading**=每进程多个线程。**Concurrency（并发）**=同一时段处理多件事（单核交错即可）；**Parallelism（并行）**=同一时刻同时做多件事（需多核）。单核两线程：并发≠并行。**线程动机**：线程另一定义=“一个可单独调度的执行序列”；是并发机制（交叠执行），也可并行（同时执行）；保护与线程正交（一个保护域可有 1 或多个线程）。进程启动时只有 1 个线程，之后通过 syscall 创建更多线程（共享地址空间）；程序员一般不直接写 syscall——**libc**（C 标准库）/OS 库和语言运行时替你调用。UNIX 结构：应用 / 标准库（用户态）→ 内核 → 硬件；中断向量表这种“按编号分派”的模式在 syscall 分派表中也出现。ComputePI();PrintClassList(); 永不打印→用两线程；掩盖 I/O 延迟（T1 阻塞 I/O 时 T2 跑）；UI 线程+后台读文件。Go **goroutine**=由语言运行时（而非 OS）调度的轻量级执行单元（简单、可扩展，放弃直接 OS 控制）。**栈帧**=一次函数调用在栈上占的区域（参数、局部变量、返回地址）。**栈追踪例**：A(int t){if(t<2)B(); printf(t);} B(){C();} C(){A(2);} 调 A(1)：栈帧 A(1,ret=exit)→B(ret=A+2)→C(ret=B+1)→A(2,ret=C+1)；输出 **2 1**。**pthread_create 内部**：库把 syscall 号放 %eax、参数放 %ebx… →trap→切内核栈存现场→查中断向量表(128)→syscall_handler 用 %eax 查 **syscall** 分派表（按 syscall 号索引内核处理函数的表）→do_fork() 建 TCB、置 READY、返回码入 %eax→恢复现场切回用户态。

4 · Pthreads 接口 (L3/L4/L7)【Sample Q1d】

```
int pthread_create(pthread_t *thread,
      const pthread_attr_t *attr,
      void *(*start_routine)(void*), void *arg);
void pthread_exit(void *value_ptr);  pthread_yield();
int pthread_join(pthread_t thread, void **value_ptr);
pthread_mutex_t m = PTHREAD_MUTEX_INITIALIZER;
/* 或 */ pthread_mutex_init(&m, NULL);
pthread_mutex_lock(&m); ... pthread_mutex_unlock(&m);
```

pthread_create=创建一个以 arg 为唯一参数执行 start_routine 的线程（函数 return 隐含调用 pthread_exit）；pthread_exit=终止调用线程，并把 value_ptr 交给成功 join 它的线程；pthread_join=挂起调用线程直到目标线程结束；pthread_yield=让出 CPU；attr=线程属性（NULL=默认）。锁（lock）=同一时刻只能被一个线程持有的对象，**mutex**=提供互斥的锁（§8）：pthread_mutex_lock=获取锁，已被他人持有则阻塞到可用；pthread_mutex_unlock=释放锁（唤醒一个等待者）；PTHREAD_MUTEX_INITIALIZER/pthread_mutex_init=静态/动态初始化锁。**Fork-join 模式**=主线程创建（fork）一批子线程分派工作，再逐个 join 收集结果。

- thread 为指针：输出参数（函数通过指针写回结果），C 按值传递，函数需把新线程 ID 写回调用者变量。
- arg 为 void*：泛型，可传任意类型/结构体地址（多参数→打包 struct）。
- pthread_exit(void*)：返回任意类型结果的地址（**勿返回本线程栈变量地址**，线程退出栈销毁→用 malloc 或全局）。
- join(void**)：结果本身是 void*，要写回调用者的 void *ret 变量→传 &ret，内部 *value_ptr = retval。
- 线程函数 return v ≡ pthread_exit(v)；**main return ⇒ 库调 exit()** 杀全进程所有线程。
- 例：create A、B 后 join：输出 main: begin → A/B 顺序不定 → main: end。join 顺序=代码顺序，但线程结束顺序/输出依赖调度，重跑可能不同。课件 pthread 例 Q&A：主线程按创建顺序 join？是；线程按创建顺序退出？否（看调度）；重跑结果会变？会；代码安全吗？不安全，线程共享变量又不加锁→**race**（竞争条件：结果取决于线程交错的时序，§8）。
- **传参 bug**：pthread_create(&t,NULL,f,&i)/&conn_fd 在循环中→子线程读到被覆盖的值（race）。修复：每次 malloc 一份或 (void*)(intptr_t)i 按值传。课件 web server 写 &arg 也是 bug（应传 arg）。

5 · 进程 API (L4)【Sample Q2.1】

exit=终止进程；fork=复制当前进程；exec=替换当前进程运行的程序；wait=等待子进程结束；kill=向进程发送信号（signal：一种类似中断的异步通知）；sigaction=设置信号处理函数。getpid()/getppid()=返回本进程/父进程的 PID。**fork()**：父得子 **PID(>0)**、子得 **0**、失败 **<0**（仍在原进程）。子进程只含一个线程（调 fork 的那个）。**逻辑复制**：整个地址空间（code/data/heap/stack，含 stdio 用户缓冲区！）、fd 表（实现用 **COW**=写时复制：父子先共享只读物理页，任一方写时才真正复制该页）。共享：fd 指向的打开文件描述（内核中代表一次 open 的对象，含当前读写位置 **offset**，§6）、终端、代码页（只读）。不共享：PID/PPID、之后的内存修改（全局变量各一份）、**pending signals**（已发出、尚未处理的信号）、文件锁。**fork 计数模板**：

- 连续/循环 n 次 fork()→2^n 进程；for(i=0;i<3;i++){fork();printf("x\n");} 打印 2+4+8=**14** 行。
- fork() && fork() || fork(); → **5** 个进程。if(fork()==0) fork();→3。fork(); if(fork()==0) fork();→6。
- printf("hi"); fork(); printf("\n"); → **hi 打印两次**（无 \n 时在用户缓冲区，被复制）；若 printf("hi\n") 输出到终端则 1 次（行缓冲：遇换行即刷出），但重定向到文件/管道（全缓冲：缓冲区满或 fflush 才刷出）仍 2 次！write(1,"hi",2) 永远 1 次。
- int x=0; if(fork()==0){x++; exit(0);} wait(NULL); printf("%d",x); → **0**。
- fork_race.c：父打印 0..9、子打印 0..-9，交错顺序不确定；加 sleep(1) 只改变可能性，不保证顺序。

exec：char *args[]={"ls","-l",NULL}; execv("/bin/ls",args); **成功永不返回**，PID 不变，fd 保持打开；之后的代码=失败处理 perror;exit(1)（perror=按 errno 打印错误原因）。变体 execl/ execlp/execvp（p=在 **PATH**（存可执行文件搜索目录的环境变量）中找程序，v=参数用数组传，l=参数逐个列出）。**wait(&status)**：阻塞到任一子进程结束，返回其 PID 并通过 status 带回退出状态；waitpid(pid,&s,0) 等指定子。**Shell 模式**（shell 运行命令的方式）：pid=fork(); if(pid==0) exec(...); else wait(&stat);。（扩展：**zombie**=子已退出但父未 wait；**orphan**=父先退出的子进程，被 **init**（系统第一个用户进程，PID 1）收养）。**main 不调 exit**：入口在 OS 库，库调 main，main 返回后库调 exit。**Syscall 接口是用户程序与内核间的“窄腰”**（所有上层程序都经这一层窄接口使用内核）：进入内核必须**原子地**（不可分割：要么整体完成，要么完全不做）同时把 PC 设到内核例程并切到内核态。线程抽象：幻觉=无限个处理器，现实=线程以不确定速度执行→程序必须对任何调度都正确。共享数据结构例（树形集合）：A Insert(3)、B Insert(4)+Get(6)，每个操作都要 acquire/release 同一把锁。信号：**SIGINT**=中断请求（Ctrl-C 发出）；**SIGTERM**=请求终止，可被捕获（kill 默认发送）；**SIGKILL**=强制杀死；**SIGTSTP**=终端暂停请求（Ctrl-Z）；**SIGSTOP**=强制暂停；每个信号有系统默认处理（SIGINT 未注册→进程死）。**SIGKILL、SIGSTOP 不能被 sigaction 改变**（保证管理员总能杀/停进程）。

```
struct sigaction sa; sa.sa_flags = 0;
sigemptyset(&sa.sa_mask); sa.sa_handler = handler;
sigaction(SIGINT, &sa, NULL);   while (1) {}
```

sa_handler=收到信号时调用的函数；sa_mask=处理期间额外屏蔽的信号集（sigemptyset 清空=不额外屏蔽）。**为何 fork+exec 两步而线程只有 pthread_create**：可不 exec 只 fork（父子代码同一可执行文件）；在 fork 后 exec 前**用代码控制子进程状态**（关/重定向 fd、改 cwd、设信号）。Windows 用 CreateProcess()（一步创建新进程并运行指定程序，参数很多）。**进程 vs 线程（讨论题）**：进程=隔离/容错（一个崩不影响他人）、安全，但创建/切换贵（换页表，**TLB**=地址翻译缓存要冲刷）、通信需 **IPC**（进程间通信机制：pipe、socket、共享内存等）；线程=创建切换快、共享内存通信方便，但无隔离（野指针/段错误拖垮整个进程）、需同步防 race。

6 · 文件与 I/O (L5)【Sample Q7.1/Q2.3】

Stream（流）=带位置的无格式字节序列（文本或二进制），C 用 FILE* 表示（用户空间结构体，含缓冲区和 fd）；**fd（文件描述符）**=open 返回的整数，是进程 fd 表的下标，只对内核有意义；**打开文件描述**（open file description）=内核中代表一个已打开文件实例的对象（记录文件数据在磁盘何处+当前位置）。层次：High-level I/O(streams, FILE*) → Low-level I/O(fd) → Syscall(open/read/ write/close) → Open File Description → File System → I/O Driver（设备驱动）→ 磁盘/Flash/DMA（直接内存访问：设备不经 CPU 直接读写内存）。**“一切皆文件”**（Ritchie & Thompson 1974）：磁盘文件、设备、socket、pipe 用同一 open/read/write/close（+ ioctl=做不适合读写模型的设备特定配置）。**POSIX**（Portable Operating System Interface）=可移植 OS 接口标准，规定标准 syscall 接口。**文件**=文件系统中命名的数据集合（POSIX 中是字节序列）+**元数据**（关于文件的信息：大小、修改时间、所有者、权限）；**目录**=包含文件和目录的“文件夹”，层次命名，路径唯一标识文件；每进程有 **CWD**（当前工作目录，chdir 修改）；**绝对路径**（/home/..，忽略 CWD）vs **相对路径**（./ ../ ~，相对 CWD）。**为何提供文件抽象(Q2.3)**：程序员——统一接口（设备无关、可移植）、命名与持久化、字节流便于编程；OS 设计者——隐藏设备细节(块/扇区)、集中做访问控制/保护、可统一缓存与调度 I/O。FILE *fopen(path, mode)=打开文件返回流，fclose(fp)=关闭流（先刷出缓冲区）。**FILE* 模式**：r 读已存在；w 写，截断/新建；a 追加，新建；r+ 读写已存在；w+ 读写，截断/新建；a+ 读写，从头读、写追加。加 b=二进制。错误返回 NULL。stdin/stdout/stderr（标准输入/输出/错误流，对应 fd 0/1/2）由系统预先打开，**可重定向**（把标准输入/输出改连到文件或管道，如 cat a | grep x、> out.txt）。**API**：fputc/fgetc=写/读一个字符（读到末尾返回 EOF）；fputs=写字符串；fgets(buf,n,fp)=读一行，最多 n−1 字符；fread/fwrite(ptr,size,n,fp)=读/写 n 个大小为 size 的元素，返回实际元素数；fprintf/fscanf=格式化写/读；fseek(fp,off,whence)=把流位置设为“起点+off”（起点 SEEK_SET=文件头、SEEK_CUR=当前位置、SEEK_END=文件尾）；ftell=返回当前位置；rewind=回到文件头；fflush=立即把用户缓冲区内容交给内核。永远检查返回值 + perror。逐字符拷贝：c=fgetc(in); while(c!=EOF){ fputc(c,out); c=fgetc(in); }（课件把 fputc 参数顺序写反了）；逐块：while((n=fread(buf,1,SZ,in))>0) fwrite(buf,1,n,out);。目录项还有 links 与 volumes（后续讲）。打开文件描述在 Linux 是 struct file（fs.h），xv6 在 file.h。**低层**：int open(path, flags[, mode])（另有 int creat(path, mode)）：flags=访问模式 O_RDONLY/ O_WRONLY/O_RDWR（只读/只写/读写）| O_CREAT（不存在则创建）/ O_TRUNC（长度截为 0）/O_APPEND（每次写都追加到末尾）；mode=权限位（user|group|other × rwx，如 0644）。返回 fd，<0 错误并设 errno（全局错误码）。ssize_t read(fd,buf,max)：返回实际字节数（**可能少于 max**），**0=EOF（到达文件尾），-1=错误**；write(fd,buf,n)=把 buf 中至多 n 字节写入 fd，返回实际写入数；close(fd)=释放该 fd 表项；lseek(fd,off,whence)=设置内核中打开文件描述的 offset（与 FILE 的位置无关）。STDIN/OUT/ ERR_FILENO=0/1/2；fileno(FILE*)=取流底层的 fd、fdopen(fd,mode)=把 fd 包装成流；dup(old)（复制出一个指向同一打开文件描述的新 fd）、dup2(old,new)（让 new 指向 old 的打开文件描述）；pipe(int fd[2])；还有文件锁、mmap（把文件映射进内存）、异步 I/O。**lowio.c**：read(fd,buf,1000) 读 min(1000, 文件大小) 字节。**为何给用户 int 而非内核指针**：内核可验证下标、用户无法伪造/篡改内核地址、内核数据结构对用户隐藏（安全+可移植）。**POSIX 模式**：使用前 open（权限检查）、面向字节（OS 隐藏块设备）、显式 close；**内核缓冲**：读时进程阻塞等设备、写交给内核后台完成即返回。

	高层 FILE* (buffered)	低层 fd (raw syscall)
位置	用户空间结构体：缓冲区 + fd	内核 fd 表下标→打开文件描述
开销	fread 先查缓冲，空时一次 read 1024B；4B 循环几乎不进内核	每次调用都陷入内核，哪怕 1 字节
可见性	printf("A ");sleep(10);printf("B\n") 一起出现	write(1,..) 立即可见
优点	减少 syscall（逐字节 read ≈10MB/s，fgetc 跟得上 SSD）；功能：fgets/ getline 按行、格式化	无延迟、精确控制字节/偏移；socket/pipe/设备；内核更简单

内核状态：每进程 fd 表 → 打开文件描述（文件在磁盘何处 + 当前 **position**，+refcnt 引用计数）。open→3; read(3,buf,100) 后 pos=100；close(3) 删 fd 项。**fork 后**：fd 被复制，打开文件描述被别名共享（aliased：两个 fd 指向同一个描述，共享 offset）：父读 100（pos 200）、子再读 100 得 200–299；一方 close 只删自己的 fd，描述在所有进程都不再引用后才释放。两次独立 open 同一文件→两个描述、独立 offset。好处：父子 printf 输出到同一终端；一方 close(0) 另一方 stdin 仍可用。

7 · Socket / Pipe / IPC (L6)【Sample Q2.2】

Socket=网络连接一个端点（endpoint）的抽象，像带 fd 的文件；连接=两个 socket 经网络相连，实现跨网络 IPC（两条队列：write 入发送队列、read 取接收队列；lseek 不可用）。4.2BSD 引入，POSIX 标准；本机或网络（**TCP**=可靠、有序的字节流协议；**UDP**=不可靠的数据报协议）。假设：可靠、有序；read 阻塞直到有数据，需循环读到终止符。字节流需上层消息/RPC。**文件当通信通道**：生产者/消费者可以是不同进程，甚至时间上分开；但“写一次读一次”更像队列→pipe/socket。Request-response：client 写请求、等、读响应；server 读请求、处理、写响应。socket 还支持过 OSI、AppleTalk、IPX（现在没人用）。**命名**：hostname → IP 地址（标识主机；v4 32 位、v6 128 位）+ 端口（port，标识主机上的通信端点/服务的 16 位编号）：0–1023 well-known（80 http、443 https、25 mail，需超级用户 bind）、1024–49151 registered（向 IANA=互联网号码分配机构 登记）、49152–65535 dynamic/ephemeral（客户端端口 OS 随机分配）。**5 元组**（源 IP、目的 IP、源端口、目的端口、协议）唯一标识连接。socket()=创建一个 socket 并返回其 fd。**Server socket**=专门等待连接的特殊 socket：有 fd 但不能 read/write；bind()=把 socket 绑定到本机地址:端口；listen()=开始允许客户端连接；accept()=为某个客户端新建一个连接 socket；connect()=客户端向服务器地址:端口 发起连接；getaddrinfo()=把主机名/端口解析成地址。

```
// client: getaddrinfo(host,port,&hints,&srv)
fd = socket(srv->ai_family, srv->ai_socktype, ..);
connect(fd, srv->ai_addr, srv->ai_addrlen);
run_client(fd); close(fd);
// server: hints.ai_flags=AI_PASSIVE, host=NULL
s = socket(..); bind(s, addr, len); listen(s, MAX_Q);
while (1) { c = accept(s, NULL, NULL);
            serve_client(c); close(c); }   close(s);
// hints: AF_UNSPEC=v4/v6 均可, SOCK_STREAM=字节流(TCP)
// AI_PASSIVE=给服务器用的本机监听地址; MAX_Q=待accept队列长
```

并发服务器=能在前一个客户端断开之前就处理新连接的服务器。**服务器演进**：v1 串行；v2 每连接 fork，子 close(s) 服务后 exit，父 close(c) 并 wait（隔离但仍串行）；v3 父不 wait→并发（子进程间仍可能经文件系统 race）；线程版：每连接 pthread_create（更快的创建/切换，无保护，共享内存 race 更多）；**线程池**=数量有界的 worker 线程集合（代表最大并发度），防网站太火时线程无限增长、吞吐崩溃（master 接受连接后 Enqueue 入队并 wakeup(queue)=唤醒睡在该队列上的线程；worker Dequeue 出队，空则 sleepOn(queue)=在该队列上睡眠）。**父必须 close(conn)**，否则 refcnt≠0 连接不会关闭且 fd 泄漏。**Pipe（管道）**=同一台机器上进程间的单向通信通道：单队列、由 pipe(fd) 临时创建、通过 fork 从父进程继承给子进程；**写 fd[1]、读 fd[0]**。父→子：父 close(fd[0]) 后写；子 close(fd[1]) 后读。**读端 EOF 仅当所有写端都关闭**（子不关自己的 fd[1] 会永远阻塞）；写满缓冲区则 write 阻塞；无读者时写→收到 SIGPIPE 信号/返回 EPIPE 错误。

```
// shell: ls | wc -l
int p[2]; pipe(p);
if (fork()==0) { dup2(p[1],1); close(p[0]);close(p[1]);
                 execlp("ls","ls",NULL); }
if (fork()==0) { dup2(p[0],0); close(p[0]);close(p[1]);
                 execlp("wc","wc","-l",NULL); }
close(p[0]); close(p[1]); wait(NULL); wait(NULL);
// shell: cmd > out.txt  (在子进程中, exec 之前)
int f = open("out.txt",O_WRONLY|O_CREAT|O_TRUNC,0644);
dup2(f, 1); close(f); execvp(cmd, argv);
```

Pipe vs socket：socket 更通用但更重（建连、IP/端口寻址、TCP 拥塞控制等），且非 **fork-native**（不能靠 fork 自然继承建立，父子需显式 connect）。**Pipe vs 文件**：文件需全局路径名、持久化、需显式同步（offset、锁、race）；pipe 短暂、fork-native。

范围	机制
同一线程	函数调用
跨线程	共享内存
父子进程	Pipe
同机无亲缘进程	UNIX domain socket / 文件(具名共享内存)
跨机器	Socket

Q2.2 无亲缘两进程：Pipe ✕（匿名，无名字，只能靠共同祖先继承 fd；具名 FIFO=有文件系统路径名的管道，才行）；Socket ✓（**UNIX domain socket**=本机内用文件路径命名的 socket；或 127.0.0.1 回环）；**Shared memory**（共享内存=多个进程把同一块物理内存映射进各自地址空间）✓（shm_open=创建/打开具名共享内存对象，mmap=把它映射进地址空间；最快，零拷贝=数据不经内核复制，但需自己加锁/信号量（§10）同步）。

8 · 并发问题与原子性 (L4/L7)

定义：Synchronization=线程间协调（常针对共享数据）；Mutual exclusion=同一时刻仅一个线程做某事；Critical section=一次只能一个线程执行的代码；Lock=同一时刻只能被一个线程持有的对象（acquire 等到空闲再标忙；release 只能由持有者调用）。**load/ store**=把一个内存字读入寄存器/把寄存器写回内存。Atomic operation=要么全做要么不做、不可分割、中间状态不可见。**多数机器上字（word，CPU 一次处理的自然长度）的 load/store 是原子的**；双精度浮点 store、数组拷贝等常非原子。**Race condition**（Sample Q1c）=结果依赖线程交错的时序。例：counter++=load; add; store，两线程各加 1000 次，结果 **∈[2,2000]**（实测 1937、1899…）。

```
int counter = 0;
void *f(void *a){ for(int i=0;i<1000;i++) counter += 1;
                  return NULL; }   // 两线程各跑一次
// 修复: lock(&m); counter += 1; unlock(&m);
```

交错枚举：A:x=1 ∥ B:y=2 → x 必为 1（独立）。A:x=y+1 ∥ B:y=2; y=y*2（y 初 0）→ **x ∈ {1,3,5}**；若 y 初 12 → x ∈ {13,3,5}。A:while(i<10)i++ ∥ B:while(i>-10)i--（仅 load/store 原子）→ 可能谁都赢不了/永不结束。（任一方都可能赢，也可能长期拉锯不结束。）ATM balance+=amount 丢失更新→给 deposit 整段加锁。**非确定性**：调度器可按任意顺序、在任意时刻切换线程。**独立线程**=不与其他线程共享状态，确定可复现；**协作线程**=与其他线程共享状态，非确定，须“设计即正确”，必须对所有交错都正确（Therac-25 放疗机的并发 bug 致死病人）。**事件驱动**=单线程循环等待下一个事件、按事件类型调用对应处理函数（§13 末尾代码）；事件驱动版 ATM 需把代码拆成非阻塞片段，线程版更自然。**Too much milk**（正确性：①最多一人买=**safety**（坏事永不发生）②需要时有人买=**liveness**（好事终将发生）；只用原子 load/store）：

- #1 先查 note 再贴：两人都查到无 note→买两份（偶发，更难调试）。#1½ 先贴 note 再查：没人买。
- #2 贴 noteA，if(!noteB) if(!milk) buy：两人同时贴→都不买（课件称 starvation=无限期等待，实为活性失败）。
- #3（课件称 Peterson；Peterson 算法=只用 load/store 实现两线程互斥的软件算法）：A leave A; while(noteB); if(!milk)buy; remove A；B leave B; if(!noteA){if(!milk)buy;} remove B。正确（假设顺序一致性：单一全局 load/store 顺序且各线程按程序序）。缺点：复杂、代码不对称（n 线程推广难，需 Lamport Bakery 面包店算法 1974）、A 忙等（busy-waiting：等待时不断空转检查、持续占用 CPU）。
- #4 用锁：acquire(&milk); if(!milk) buy; release(&milk);

经典 Peterson（2 线程）（flag[i]=线程 i 想进入；turn=轮到谁让步；CS=临界区）：flag[i]=1; turn=j; while(flag[j]&&turn==j); CS; flag[i]=0;

9 · 锁的实现 (L8/L13)

方案 1 关中断（屏蔽中断，使当前代码不会被切走）：调度器拿回 CPU 只有两条路：内部事件（线程主动放弃）和外部事件（中断）。单核上避免两者（不主动让出+关中断）就不会切换。朴素 Acquire{关中断} Release{开中断}：用户代码死循环可锁死系统、临界区任意长、实时/I/O 事件丢失。**聪明版**：用 value 变量表示锁状态，只在操作锁变量时关中断；拿不到锁就进等待队列（wait queue，因等待某事件而睡眠的线程队列）并 sleep()（主动放弃 CPU、进入 waiting/blocked 状态）：

```
Acquire() { disable ints;
  if (value == BUSY) { put on wait queue;
                       go to sleep(); /* 开中断? */ }
  else value = BUSY;
  enable ints; }
Release() { disable ints;
  if (anyone on wait queue) { take off; make READY; }
  else value = FREE;   // 有等待者时锁直接“交接”
  enable ints; }
```

sleep 前在何处开中断？丢失唤醒（missed wakeup）=唤醒信号在线程真正睡下之前到来而被错过，导致它永远睡下去。入队前开→释放者看不到等待者→丢失唤醒；入队后 sleep 前开→被唤醒后又去睡→丢失唤醒；**正确：关着中断进入 sleep，由切换到的下一线程负责开中断**（被唤醒线程从 sleep 返回后开）。总有 **idle thread**（开机时创建、永不阻塞的空闲线程）可切换。缺点：不能给用户用、多核无效。**原子读改写指令**（以下每条指令都由硬件整体原子执行；read-modify-write：原子地读出一个值并写入新值；单核/多核均可，多核需缓存一致性协议=保证各核缓存中同一数据一致的硬件机制）：

```
test&set(&a) { r=M[a]; M[a]=1; return r; } // 多数架构
swap(&a, reg) { t=M[a]; M[a]=reg; reg=t; }  // x86
compare&swap(&a, r1, r2) {             // x86, 68000
  if (M[a]==r1) { M[a]=r2; return success; }
  return failure; }
add_to_queue(&obj) {  // 无锁(lock-free)入队: 不用锁, CAS 失败就重试
  do { r1 = M[root]; M[obj].next = r1; }
  until (compare&swap(&root, r1, obj)); }
acquire(int *l) { while (test&set(l)); }   // spin
release(int *l) { *l = 0; }
// CAS 原子加: do{old=x;}while(!CAS(&x,old,old+1));
```

自旋锁（spinlock）=拿不到锁时忙等而不睡眠的锁。**T&S 自旋**：+可在用户态用、可响应中断、多核可用；−忙等浪费周期、可能抢走持锁线程的 CPU（单核更糟）、每次 T&S 都是写→缓存行在核间来回失效搬运（**ping-pong**），占用大量总线带宽。考试答案应避免忙等！**T&T&S（test-and-test-and-set，L13）**：do{ while(value); } while(test&set(&value)); 先只读本地缓存自旋，看到空闲才原子写→消除 ping-pong。自旋锁适用：多核上短等待（如 **barrier**=所有线程都到达某点后才一起继续），比进内核睡眠/唤醒更快。**guard 锁**：用一个 guard 变量保护锁状态，只在检查/修改锁状态时短暂自旋，拿不到真正的锁就睡眠：

```
acquire(int *l) { while (test&set(guard));  // 短忙等
  if (*l == BUSY) { put on wait queue;
       go to sleep() & guard = 0; }  // 原子完成!
  else { *l = BUSY; guard = 0; } }   // 醒来时 guard==0
release(int *l) { while (test&set(guard));
  if (anyone waiting) { take off; place on ready q; }
  else *l = FREE;
  guard = 0; }
```

futex（fast userspace mutex）=Linux 提供的“有条件睡眠”内核接口，无竞争时在用户态完成加锁，需要等待时才进内核睡（int futex(int*uaddr,int op,int val,timespec*to)）：FUTEX_WAIT：若 *uaddr==val 则睡（内核中原子检查，避免丢失唤醒）；FUTEX_WAKE：唤醒至多 val 个。不在 libc 暴露，用于实现 pthread 锁/信号量/管程（§10–11）。T&S+futex：acquire{while(test&set(l)) futex(l,WAIT,1);} release{*l=0; futex(l,WAKE,1);}：无忙等、获取无内核开销，但每次 **unlock** 都进内核（即使无等待者，可加 waiters 计数优化）。**trylock**=尝试获取锁，失败立即返回而不阻塞。硬件锁指令（Intel 432）使硬件复杂变慢，不采用。

10 · 信号量 Semaphores (L8/L9)【Sample Q3】

信号量（semaphore）=一种广义的锁：一个非负整数加两个原子操作（Dijkstra 60 年代末提出，原始 UNIX 主原语）；**P()**（荷兰语 proberen“测试”，又名 down/wait）=原子操作：等待信号量值变为正数，然后将其减 1（值为 0 时调用者阻塞睡眠）；**V()**（verhogen“增加”，又名 up/signal/post）=原子操作：将信号量值加 1，若有线程阻塞在 P 上则唤醒其中一个。除初始化外**不能直接读写值**；两个操作都原子（两个 P 不会把值减到负；P 睡下不会错过 V 的唤醒）。**用途**：初值 1=互斥（**binary semaphore**（即 mutex，只取 0/1））；初值 0=**调度约束**（规定线程间先后顺序，让一个线程等另一个的信号）（s 初值 0：ThreadJoin{P(s)} 等待，ThreadFinish{V(s)} 放行）。**经验法则：每个约束一个信号量。**

```
// Q3.1 bounded buffer(有界缓冲区): empty=N, full=0, mutex m
void put(Item x) {             Item get() { Item x;
  wait(empty); //①先             wait(full); //①先
  lock(m);     //②后             lock(m);    //②后
  buf[tail] = x;                 x = buf[head];
  tail = (tail+1) % N;           head = (head+1) % N;
  unlock(m);                     unlock(m);
  signal(full); }                signal(empty);return x;}
```

- **Q3.2 去掉 mutex**：❌（N>1 且多生产者/消费者）。例：P1、P2 都通过 wait(empty)，同时读 tail=3，都写 buf[3]→一个元素被覆盖丢失、tail 只加 1 或错乱，消费者读到重复/垃圾。（单生产者+单消费者时可以，因为 head/tail 各自只被一方修改。）
- **Q3.3 交换 wait(empty) 与 lock(m)**：**死锁**（一组线程循环等待彼此持有的资源，谁都无法继续）。缓冲满时生产者持 m 睡在 empty 上，消费者卡在 lock(m) 无法取走元素腾位→互相等待。（交换两个 V/signal 的顺序无害，只影响效率。）
- **Q3.4 缺锁的 put**：缺互斥保护 buf/tail；问题：两生产者写同一槽位丢数据、tail 更新丢失。
- 不对称：生产者 P(empty) V(full)；消费者 P(full) V(empty)。缺点：信号量一身二用（互斥+调度），正确性（如 P 顺序致死锁）不直观→引出管程。

常考模式（**rendezvous** 汇合=两线程都到达后才各自继续；**turnstile**=P(s);V(s) 闸门：一次放一个线程通过，再把门留给下一个）：

```
// A 先于 B: s=0;  T1: A; V(s);   T2: P(s); B;
// rendezvous: a=b=0; T1: V(a);P(b);  T2: V(b);P(a);
// n 线程 barrier: mutex=1, cnt=0, bar=0
P(mutex); cnt++; if (cnt==n) V(bar); V(mutex);
P(bar); V(bar);                            // turnstile
// 用 lock + CV(条件变量,§11) 实现信号量
P() { lock(m); while (v==0) wait(&cv,&m);
      v--; unlock(m); }
V() { lock(m); v++; signal(&cv); unlock(m); }
```

Circular buffer 错误版：（**生产者-消费者/有界缓冲区问题**=生产者往固定大小的共享缓冲区放数据、消费者取数据；满则生产者等，空则消费者等，缓冲区让两者不必步调一致。例：GCC 流水线 cpp | cc1 | cc2 | as | ld、可乐机、web 服务器、路由器。缓冲区结构：write_index, read_index, entries[BUFSIZE]；满/空判定和入队/出队要原子。）持锁 while(full){}→永远出不来（对方拿不到锁）；while(full){release;acquire;}→能工作但忙等（单核尤其浪费）。

11 · 管程 Monitors 与条件变量 (L9)

Monitor（管程）=1 把锁 + 0 或多个条件变量，用于管理对共享数据的并发访问的编程范式（Java 原生，C 用锁+CV 对象）。锁→互斥；CV→调度约束。**CV（条件变量）**=临界区内等待某条件成立的线程队列；关键：可在临界区内睡眠，睡时原子释放锁（信号量不能在临界区内等待）。wait(&lock)（pthread 中为 cond_wait）=原子地释放锁并睡眠，被唤醒后返回前重新获取锁；signal()=唤醒一个等待者（若有）；broadcast()=唤醒所有等待者；**做 CV 操作必须持锁**。CV 无记忆：无人等待时 signal 是空操作（与信号量 V 不同）。**Hoare 语义**：signaler（发信号者）立即把锁和 CPU 交给 waiter（被唤醒者），waiter 退出临界区/再 wait 时交回→if 即可。**Mesa 语义**（Xerox PARC，大多数 OS）：signaler 保留锁和 CPU，waiter 只进 ready 队列无特权，别的线程可能先“插队”改状态→**必须 while 重检**（这不是忙等，因为在睡眠）。

```
Producer(item) { acquire(&L);
  while (full) cond_wait(&not_full, &L);
  enqueue(item); cond_signal(&not_empty);
  release(&L); }
Consumer() { acquire(&L);
  while (empty) cond_wait(&not_empty, &L);
  item = dequeue(); cond_signal(&not_full);
  release(&L); return item; }
```

Mesa 程序结构：lock; while(need wait) cv.wait(); unlock; 做事; lock; 更新状态; cv.signal(); unlock; **读者-写者问题**=共享数据库：读者只读、写者读写；允许多个读者同时读，但写者必须独占。**写者优先解法**：状态 AR/WR/AW/WW=活跃读者/等待读者/活跃写者/等待写者数（初值 0），CV ok_to_read/ok_to_write。

```
Reader() { acquire(&L);
  while (AW+WW > 0) { WR++; cond_wait(&okR,&L); WR--; }
  AR++; release(&L);
  read_db();
  acquire(&L); AR--;
  if (AR==0 && WW>0) cond_signal(&okW);
  release(&L); }
Writer() { acquire(&L);
  while (AW+AR > 0) { WW++; cond_wait(&okW,&L); WW--; }
  AW++; release(&L);
  write_db();
  acquire(&L); AW--;
  if (WW > 0) cond_signal(&okW);          // 写者优先
  else if (WR > 0) cond_broadcast(&okR);
  release(&L); }
```

追踪 R1,R2,W1,R3：R1、R2 读(AR=2)；W1 等(WW=1)；R3 因 WW>0 等(WR=1)；R2 完 AR=1；R1 完 AR=0 且 WW>0→signal W1；W1 写(AW=1,WW=0)；W1 完：WW=0、WR>0→broadcast；R3 读。**问答**：读者会饿死吗？会（写者源源不断）。删掉读者退出时的 AR==0&&WW>0 检查？仍正确（写者被唤醒后 while 重检再睡），只是多余唤醒。signal 改 broadcast？仍正确，效率低。只用一个 CV？必须 broadcast 才正确（signal 可能唤醒错误类型线程而丢失唤醒）。**语言支持**：C 中每条退出路径都要 release（多锁更难）；异常非局部退出会漏释放→try/catch 释放后重抛；C++ std::lock_guard<std::mutex> g(m);（构造时加锁、析构时自动释放）；Python with lock:；Java synchronized 方法（进入方法时自动获取对象锁、退出时释放，异常时也释放），每个对象一把锁+一个 CV：wait()/wait(timeout)=等待，notify()/notifyAll()=唤醒一个/全部。**同步层次**：硬件（load/store、关中断、test&set、compare&swap）→ 高层 API（锁、信号量、管程、send/receive=消息传递原语）→ 共享程序。

12 · 简答模板（Side A 范围）

- **Process vs thread**：进程=地址空间+线程（保护单位）；线程=执行上下文（调度/并发单位）。线程优点：创建切换便宜、共享数据通信方便；缺点：无隔离、需同步。
- **为何双模式**：防止用户程序直接改页表/关中断/访问设备/内核内存，保护 OS 与他进程。
- **抢占需要**：硬件定时器中断（否则协作式死循环卡死）。
- **用户态缓冲 pros/cons**：+少 syscall、行/格式化功能；−延迟可见性（崩溃丢数据、与 write 混用顺序错乱、fork 复制缓冲区）。
- **锁三要素检查**：互斥；**进展**（临界区空闲时，想进入的线程最终能进入=无死锁）；**有限等待**（发出请求后，别人插队进入的次数有上限=无饥饿）；避免忙等；所有路径释放。
- **fd 追踪题**：①open 返回**最小可用 fd**：close(1); open("f",..) → 得 1，之后 printf 写入文件（老式重定向）。②先 open 再 fork，父写 “AB”、子写 “CD”→共享 offset，文件含 4 字节（ABCD 或 CDAB，可能交错）；fork 后各自 open→各自 offset=0，**互相覆盖**，只剩 2 字节。③dup2(fd,1) 后 printf 进文件（记得 fflush）。④读到文件尾后 read 返回 0。
- **线程/进程计数**：pid=fork(); if(pid==0){fork(); pthread_create(..);} → 进程 3 个（父、子、孙），新建线程 2 个（子、孙各一个）。
- **判断锁实现是否正确**：构造交错：两线程都在“检查”后、“设置”前被切换→都进入临界区⇒违反互斥；或都在等对方⇒死锁/活锁。

13 · 课件原代码与细节补遗 (L2–L9)

执行循环（von Neumann 结构：指令与数据存于同一内存）：取指(PC) → 译码 → 执行(用寄存器) → 写回寄存器/内存 → PC=下一条 → 重复。**B&B 装载进程**：OS(sysmode=1) 把 P 的 code/data 放到 1000…；设置 Base=1000、Bound=1100、uPC=0000（特权指令）；RTU → sysmode=0, PC=uPC，用户代码运行，地址经 Base 翻译。回到内核：定时器中断、I/O 请求、syscall 等。线程状态图：

```
New -admit-> Ready -dispatch-> Running -exit-> Term
             ^  ^<-preempt/yield--|   |
             |                        | 等 I/O/event
             +--完成-- Waiting/Blocked <-+
```

Too much milk 原代码：

```
// #1                          // #1½ (没人买)
if (noMilk) {                  leave Note;
  if (noNote) {                if (noMilk) {
    leave Note; buy milk;        if (noNote) buy milk;
    remove Note; } }           } remove Note;
// #2 (可能都不买)              // #3 (正确, A 忙等)
leave noteA;                   leave noteA;      // A
if (noNoteB) {                 while (noteB) ;   // X
  if (noMilk) buy milk; }      if (noMilk) buy milk;
remove noteA;                  remove noteA;
// #3 B: leave noteB; if (noNoteA) { if (noMilk) buy; }
//       remove noteB;            (if 处为 Y)
```

3 正确性：X 处若无 noteB，A 可安全买；否则等 B 结果。Y 处若无 noteA，B 可安全买；否则 A 要么在买要么在等 B 退出。分两种情况（leave noteA 先于/后于 B 的 if(noNoteA)）讨论即可证明。**枚举所有可能输出（技巧）**：把每个线程拆成原子步（load/ store/print），保持各线程内顺序，列举交错；只关心影响结果的步骤。例 main 创建 T1 打印 “A”、T2 打印 “B” 再 join：输出 begin A B end 或 begin B A end；若不 **join** 就 return：A/B 可能根本不打印（main return⇒exit 杀死线程）。

CSC3150 OS Midterm · Side B：AI 系统 / 调度 / 公平 / 实时 / 死锁 / 集群（L10–L14）

计算题：Gantt → 完成时刻 → 公式 → 平均

14 · 调度基础与公式 (L11)

调度=决定哪个线程何时获得资源（CPU、网络带宽、磁盘）。程序交替 **CPU burst**（连续使用 CPU 的一段时间）与 I/O burst（等待 I/O 的一段时间），burst 分布偏向短 burst；每次决策=把 CPU 给谁跑下一个 burst；**时间片**（time quantum q）=一个线程一次最多连续运行的时间，可迫使线程在 burst 结束前让出。**Gantt 图**=按时间轴画出每段时间由哪个作业占用 CPU。**公式：周转/完成时间**（从到达到完成）$= T_{\text{finish}} - T_{\text{arrival}}$；**等待时间**（在就绪队列中等待的总时间）= 周转 − CPU − (I/O，若有)；**响应时间**（从到达到第一次运行）$= T_{\text{first run}} - T_{\text{arrival}}$；**吞吐**=完成作业数/时间；**CPU 利用率**=CPU 忙的时间/总时间。**课件“completion time”常从 0 算（所有作业 0 时刻到达）**。**目标**：①最小化响应时间（用户可见：回显、编译；实时任务须满足截止期=必须完成的时刻）②最大化吞吐（最小化开销如上下文切换 + 高效利用 CPU/磁盘/内存；与响应时间相关但不同——最小化响应时间需更多切换）③公平（公平 ≠ 最小平均响应时间；更不公平反而平均响应更好）④可预测性。**评估方法 (L13)**：**确定性建模**=取一个预定负载计算各算法表现；**排队模型**=用数学方法处理随机负载；**实现/模拟**=在真实数据上运行真实算法（最灵活）。

15 · FCFS / RR / SJF / SRTF (L11)

非抢占=线程一直运行到完成或主动阻塞；**抢占**=OS 可中途收回 CPU。**FCFS/FIFO**（先来先服务）=按到达顺序运行，早期指运行到结束，现指保持 CPU 直到阻塞（非抢占）：P1=24,P2=3,P3=3 均 0 到：顺序 1,2,3 → 等待 0,24,27 平均 **17**，完成 24,27,30 平均 **27**；顺序 2,3,1 → 等待 6,0,3 平均 **3**，完成平均 **13**。**Convoy effect（护航效应）**=短作业卡在长作业后面等待。+简单 −依赖到达顺序。**RR（轮转）**=每个进程得到一个小的 CPU 时间片 q（10–100ms），到期被**定时器中断抢占**并放到就绪队列尾；n 个就绪→每个得 1/n CPU、每段≤q、**等待间隔≤(n−1)q**。q→∞ 退化 FCFS；q 太小切换开销占满（吞吐崩）；q 须≫切换开销（0.1–1ms）→ 典型 10–100ms，≈1% 开销。早期 UNIX q=1s：3 个编译时回显等 3s。+短作业好、公平；−长作业累计切换开销。**RR q=20**：P1=53,P2=8,P3=68,P4=24：`P1 0-20|P2 20-28|P3 28-48|P4 48-68|P1 68-88|P3 88-108|P4 108-112|P1 112-125|P3 125-145|P3 145-153`。等待 P1=72,P2=20,P3=85,P4=88 平均 **66.25**；完成 125,28,153,112 平均 **104.25**。

Wait/Comp	P1	P2	P3	P4	**Avg**
Best FCFS	32/85	0/8	85/153	8/32	31¼/69½
Q=1	84/137	22/30	85/153	57/81	62/100½
Q=5	82/135	20/28	85/153	58/82	61¼/99½
Q=8	80/133	8/16	85/153	56/80	57¼/95½
Q=10	82/135	10/18	85/153	68/92	61¼/99½
Q=20	72/125	20/28	85/153	88/112	66¼/104½
Worst FCFS	68/121	145/153	0/68	121/145	83½/121¾

RR 不总比 FCFS 好：10 个 100s 作业同时到，q=1s，零切换开销：FCFS 完成 100,200,…,1000（平均 550）；RR 991,992,…,1000（平均 995.5），同时全部结束。**等长作业时 FCFS 最优**。**SJF**（最短作业优先，=STCF，非抢占）=运行计算量最少的作业 / **SRTF**（最短剩余时间优先，=SRTCF）=SJF 的抢占版：新到作业剩余时间比当前作业剩余时间短就立即抢占：可用于整程序或当前 burst。**可证最优**：SJF 在非抢占中、SRTF 在抢占中平均响应时间最小（SRTF≥SJF，故关注 SRTF）。作业等长时 SRTF=FCFS。+最优；−需预知未来、**饥饿**（starvation：线程无限期得不到资源；大量短作业时长作业永不运行）、不公平。可作为衡量其他策略的标尺。预测（**自适应**=根据过去行为调整策略）：让用户申报（超时杀掉防作弊）或**指数平均**（用历史 burst 加权估计下一个 burst）$\tau_n = \alpha t_{n-1} + (1-\alpha)\tau_{n-1}$，$0 < \alpha < 1$（例 α=0.5, τ_0 =10, 实际 6,4,6,4 → 预测 8,6,6,5）。**SRTF 磁盘例**：A、B 计算一周，C 循环 1ms CPU+9ms 磁盘。单独跑 C 用 90% 磁盘。FCFS：A/B 霸占两周，磁盘≈0；RR 100ms：每 201ms 磁盘忙 9ms→**9/201≈4.5%**；RR 1ms：≈90% 但大量唤醒/切换；SRTF：C 一就绪就抢占→**90%** 且 A/B 不被无谓切换。**带到达时间的通用算例**（自练）：P1(到 0,8) P2(1,4) P3(2,9) P4(3,5)：

- FCFS：P1 0-8,P2 8-12,P3 12-21,P4 21-26；等待 0,7,10,18 → **8.75**。
- SJF 非抢占：P1 0-8,P2 8-12,P4 12-17,P3 17-26；等待 0,7,15,9 → **7.75**。
- SRTF：P1 0-1,P2 1-5,P4 5-10,P1 10-17,P3 17-26；等待 9,0,15,2 → **6.5**。
- RR q=4：P1 0-4,P2 4-8,P3 8-12,P4 12-16,P1 16-20,P3 20-24,P4 24-25,P3 25-26；完成 20,8,26,25；等待 12,3,15,17 → **11.75**。（约定：同一时刻新到达者先于被抢占者入队尾。）

严格优先级=总是把最高优先级的可运行作业运行到完成，同级 RR。问题：饥饿（低优先级永不运行）、**优先级反转**（低优先级任务持有高优先级任务需要的锁，§19）。修复：**动态优先级**=按交互性、持锁、burst 行为等启发式上下调整基础优先级。公平：严格优先级不公平；给每队列固定 CPU 份额（1 长 100 短时像超市快速通道变长）、提高久未服务作业的优先级（ad hoc；过载时人人升级→交互作业受害）。**公平以牺牲平均响应时间为代价。**

16 · MLFQ 与 Linux O(1) (L12)【Sample Q4】

MLFQ（多级反馈队列）=利用过去行为的调度：多个不同优先级的队列（高=**前台交互作业**，低=**后台批处理作业**），作业根据行为在队列间升降，各队列有自己的算法（前台 RR、后台 FCFS；或 RR 时间片指数递增 1,2,4ms）。规则：作业从最高队列开始；**用完时间片→降一级**；未用完（I/O 让出）→**保持或升一级**。结果近似 **SRTF**：**CPU** 密集作业（大部分时间在计算）“像石头一样沉底”，短的 **I/O** 密集作业（频繁等 I/O、burst 短）留在顶部。队列间调度：固定优先级（先服务完高队列）或按时间片分配（70%/20%/10%）。**Countermeasure（反制）**=用户为钻空子而破坏 OS 设计意图的做法：插入无意义 I/O 保持高优先级——Othello 程序加 `printf` 跑得更快；人人都这么做就失效。防御：按每级累计 CPU 时间计配额、定期 **priority boost**（把所有作业移回最高队列，防饥饿）。**调度器的隐含假设**：经常睡眠、burst 短=交互式→高优先级；计算多=低优先级（它们不在意偶尔被交互任务打断）。难点：有的程序长睡后又长算，有的必须在任何情况下（如周期性）运行。**Q4.3 偏向交互作业**（交互作业=需要快速响应用户的短 burst 作业）**的特性**：主动让出（阻塞 I/O）的作业不降级、留在高优先级短时间片队列，且高队列抢占低队列→响应快。**Q4.4 MLFQ 表现差的场景**：①长 CPU 作业在大量交互作业下饥饿（无 boost）②**博弈**：时间片用完前故意 I/O 霸占高优先级 ③**行为变化**：先计算后转交互的作业被困低优先级 ④参数（队列数、q、boost 周期）难调。**Sample Q4 追踪**：Q0(q=2)、Q1(q=3)、Q2 FCFS，抢占，无 boost；A 到 0 需 6；B 到 1 需 3；C 到 2：1 CPU→2 I/O→1 CPU→2 I/O→1 CPU。约定：被抢占作业保留在本级已用配额并回到队首。

t	**Job**	**Q**	**事件**
0–1	A	Q0	t=1 B 到达入 Q0
1–2	A	Q0	**t=2 A 用满 2→降 Q1**（剩 4）；C 到达排在 B 后
2–3	B	Q0	
3–4	B	Q0	**t=4 B 降 Q1**（剩 1，排 A 后）
4–5	C	Q0	t=5 C 阻塞 I/O [5,7)，留 Q0
5–6	A	Q1	A 在 Q1 用 1/3
6–7	A	Q1	用 2/3；**t=7 C 返回 Q0 抢占 A**
7–8	C	Q0	t=8 C 再阻塞 [8,10)
8–9	A	Q1	**t=9 A 用满 3→降 Q2**（剩 1）
9–10	B	Q1	**t=10 B 完成**；**t=10 C 返回**
10–11	C	Q0	**t=11 C 完成**
11–12	A	Q2	**t=12 A 完成**

降级：t=2(A), 4(B), 9(A)；C 返回：t=7, 10；完成 B=10, C=11, A=12；等待 A=12−0−6=**6**，B=10−1−3=**6**，C=11−2−3−4=**2**。**Linux O(1)**：基于优先级、所有操作 O(1) 的调度器；140 优先级：**0–99 内核/实时，100–139 用户**（由 **nice** 值设定：用户可调的谦让值），**数值小=优先级高**；所有操作 O(1)（时间片/优先级/交互信用在时间片用完时计算；140 位位图找非空队列）。**active/expired** 两数组（各 140 个队列）：active=时间片还没用完的任务，expired=时间片已用完的任务；用完时间片移入 expired，active 空则两者交换。时间片随优先级线性映射，按“Timeslice Granularity”（同优先级任务间轮转的粒度）分块在同优先级 RR。启发式：用户任务优先级 ±5，`sleep_avg = sleep_time − run_time`（越大越 I/O 型→奖励）；**Interactive Credit**（交互信用）：长睡赚、长跑花，提供滞后（hysteresis：不因短暂行为变化而立刻改变交互判定）；交互任务用完时间片放回 active（除非他任务饿太久）。多核：每核一套队列+后台再平衡。

17 · 公平共享 / Lottery / WFQ / EEVDF (L12)

公平=两个相同的进程应得到相同的服务、相同的性能；**work-conserving**=有工作可做时不让 CPU 空闲；公平避免饥饿（例 RR）。源于网络（Fair Queueing, SIGCOMM’89），后用于 CPU（Lottery, OSDI’94）。**Max-min fairness**=最大化最小分配：需求小于公平份额的作业先得到其全部需求，剩余容量在其他作业间均分（每个作业保证拿到其公平份额=隔离）：

```
// C=剩余容量, S=未满足作业集, D=需求, A=分配, f=公平份额
while S≠∅: f=C/|S|; b=true
  for Ji in S: if D[i] <= f:
      b=false; A[i]=D[i]; C-=D[i]; S-=Ji
  if b: for Ji in S: A[i]=f; S-=Ji
```

例 C=10：D=[8,6]→[5,5]；[8,4]→[6,4]；[8,6,2]：f=10/3，J3 得 2，C=8，f=4→**[4,4,2]**。等价求 f（水位）使 $\sum_i \min(r_i, f) = C$（r_i=作业 i 的需求）。加权 **max-min**=按重要性给作业权重，份额按权重分：$\sum_i \min(r_i, fw_i) = C$（$w_i$=权重）。例 r=[8,6,2], w=[3,1,1]→f=2：min(8,6)=6, min(6,2)=2, min(2,2)=2→**[6,2,2]**。解法：按 r_i/w_i 升序逐个“注水”。课件图：3 个作业平分各 33%；作业 1 只要 ≤20%→20/40/40；权重 1:2→33%/66%。用途：比例份额、优先级（权重 1000:1）、预留（权重 10，总和≤100→10%）、截止期调度（由需求与截止期算权重）、隔离（不能影响他人份额之外）。广泛使用：Lottery、Linux **CFS**（完全公平调度器：按权重累计 vruntime 选最小者）；WFQ 及其变体 WF²Q/SFQ/DRR/CSFQ（各种公平队列算法）；Hadoop fair scheduler、**DRF**（主导资源公平：多资源的 max-min）。**Lottery scheduling（彩票调度）**=加权公平共享的概率近似：每个作业持有与权重成正比的票数，每次随机抽一张票决定谁运行；$N_{\text{ticket}} = \sum N_i$，随机 d ∈ [1,N]，选前缀和首次≥d 的作业（按 N_i 排序）。短作业 10 票、长作业 1 票：

# 短/# 长	每个短	每个长
1/1	91%	9%
0/2	N/A	50%
2/0	50%	N/A
10/1	9.9%	0.99%
1/10	50%	5%

公式：短 = 10/(10·# 短+# 长)。负载过高（load 100）时都难进展→注销用户。+无饥饿（票数>0 就有机会）、简单、易增删。**流**（flow）=来自同一发送者的一串包；**流积压**（backlogged）=有包在排队等待发送。**GPS（广义处理器共享）**=理想化的流体模型：资源可无限细分，所有积压的流按权重同时被服务（逐比特加权 RR，每轮每流发 w_i 比特）；**WFQ（加权公平队列）**=用分组系统近似 GPS：每次发送在 GPS 中最早完成的包（假设无后续到达）。新到达会改变完成时刻但不改变完成顺序→用**虚拟时间**（系统虚拟时间 V(t)=逐比特轮转中当前的轮次编号；包的虚拟完成时间=发完该包需要的轮数）：$\frac{\partial V}{\partial t} = \frac{C}{N(t)}$（$N(t)$=活跃流总权重；活跃流越多 V 增长越慢）。包虚拟完成时间：$F_i^{k+1} = \max\left(V\left(a_i^{k+1}\right), F_i^k\right) + \frac{L_i^{k+1}}{w_i}$（$a_i^k$=流 i 第 k 个包的到达时刻，$L_i^k$=其长度）。例：A(w=2)、B(w=1) 持续积压，包长 1：F_A =0.5,1,1.5,2…，F_B=1,2,3…→A 每轮发 2 个、B 1 个。课件 GPS 例：一个权重 5 的流和五个权重 1 的流都积压，每轮发 10 bit（5,1,1,1,1,1）。V(t) 例：只有 1 个流积压时斜率 C，2 个等权流积压时 C/2。WFQ 实例：新包在 t=ε 到达会改变完成时刻（需重算），但不改变完成顺序。EEVDF 课件例：红流占 1/2 容量，WFQ 下得到 5、流体系统中只有 2.5（lag −2.5）；理想是每隔一个时隙发一个红包。**EEVDF**（Earliest Eligible Virtual Deadline First，Linux 6.6+ 调度器）=只在合格任务中选**虚拟截止期**（虚拟开始时间+请求量/权重）最早者。**lag**=任务在理想流体系统中应得服务与实际所得服务之差。WFQ 可让一个流连续发多个包而某流（如占一半容量的红流）被拒 n 个时隙，**lag 最坏 O(n)**。EEVDF：只调度合格（虚拟开始时间 ≤ V(t)，即 lag≥0）的任务中虚拟截止期最早者→**lag≤0.5 个量子（量子=一次调度的时间片），与流数无关**。

算法	错过截止期	Lag	复杂度
Fair queueing	q	O(n)q	O(log n) 优先队列
EEVDF	q	**O(1)q**	O(log n) 更复杂
Lottery	O(√n)q	O(√n)q	O(1)
Round robin	q	O(n)q	O(n) 查空队列；难处理不同量子

18 · 实时调度 / 多核 / 选型 (L13)

实时调度=以可预测性为目标：能事先有把握地给出最坏情况响应时间；实时≠计算快。传统系统面向吞吐、事后观察。**Hard RT（硬实时）**=用于时间关键、安全攸关的系统，必须满足所有截止期，最好事先判定能否满足：EDF、**LLF**（最小松弛度优先，松弛度=截止期−当前时间−剩余计算量）、**RMS**（速率单调：周期越短优先级越高，静态）、**DM**（截止期单调：相对截止期越短优先级越高，静态）。**Soft RT（软实时）**=用于多媒体，尽量以高概率满足截止期：**CBS**（恒定带宽服务器：给每个软实时任务预留固定比例的 CPU 带宽）。模型：任务可抢占、独立、任意到达，已知截止期 D 与计算时间 C（**绝对截止期**=具体时刻；**相对截止期**=从到达起算的时长）。**RR 不适合**（会错过截止期）。**EDF（最早截止期优先）**=按绝对截止期远近给任务动态分配优先级，总是调度截止期最近的活跃任务。周期任务 (P_i, C_i)（周期 P、每周期计算量 C），抢占式**动态优先级**：绝对截止期 $D^{t+1} = D^t + P_i$，总调度最近截止期者。**可行性**（存在一种调度使所有任务都满足截止期）：$\boldsymbol{U = \sum_{i=1}^{n} \frac{C_i}{D_i} \leq 1}$（U=CPU 利用率）。例 (C,P)=(1,4),(2,6),(3,8)：U=0.25+0.333+0.375=0.958≤1 ✓。（RMS 充分条件 U≤ $n\left(2^{\frac{1}{n}} - 1\right)$：n=3 为 0.780→该例 RMS 不保证。）**多核**：算法与单核差别不大；实现上每核数据结构（缓存一致性）；**亲和性调度**（affinity）=线程在某 CPU 上运行过后，OS 尽量继续把它调度到同一 CPU（缓存复用、分支预测）。自旋锁/T&T&S 见 Side A §9。

关心	选择
CPU 吞吐	FCFS
平均响应时间	SRTF 近似
I/O 吞吐	SRTF 近似
公平（CPU 时间）	RR、Fair sharing、Lottery、EEVDF
满足截止期	EDF
重要任务优先	Priority

何时买更快硬件：资源不足时策略才重要；利用率→100% 时响应时间→∞；多数调度在负载曲线线性段表现良好，到达拐点**（knee，响应时间开始急剧上升处）**就该升级。

19 · 优先级反转 (L13)

优先级反转=高优先级任务被阻塞、等待低优先级任务（中优先级任务因此可饿死高优先级任务）。低优先级 J1 持锁；高优先级 J3 请求该锁阻塞；中优先级 J2 抢占 J1 一直运行→J3 被 J2 间接饿死。另例：高优先级 `while(try_lock)` 自旋（trylock 见 §9），低优先级持锁永远得不到 CPU。**解法：优先级捐赠/继承**=高优先级任务把自己的优先级临时授予持锁者，让它代自己运行：J3 阻塞时把高优先级临时借给 J1，J1 释放锁后恢复，J3 获锁运行（调度器须知道谁持锁）。**Mars Pathfinder (1997, 实时 OS VxWorks)**：低优先级 ASI/MET 采集任务持互斥锁，高优先级数据分发任务需要它，中优先级任务长期占 CPU→**看门狗**（watchdog：检测到系统无进展就复位的机制）反复重启；原开发者为性能关掉了优先级继承，远程上传补丁重新打开解决。

20 · 死锁 (L13)【Sample Q5】

Starvation=无限等待（低优先级被高优先级持续占用资源），可能自行结束；**Deadlock**=循环等待资源，无外部干预无法结束。**死锁⇒饥饿，反之不成立**。例：单车道桥（各占一半等另一半；倒车=抢占+回滚；东行车流太快西行永远过不去=饥饿非死锁）；两线程 x→y vs y→x 加锁（**非确定**，有时死锁有时不，难调试）；内存 2MB 两线程各 `AllocateOrWait(1MB)` 两次；还可死锁于终端、打印机、内存、pipe、socket 等。**Dining lawyers**（哲学家就餐变体）：5 人 5 筷子，吃饭需两根，各拿一根→死锁；预防：若拿走最后一根后无人能有两根则不许拿（k 只手：最后一根且无人凑齐 k、倒数第二根且无人有 k−1…）。**四必要条件**：①**互斥**（资源一次一线程）②**持有并等待**（持有≥1 资源又等别人的）③**不可抢占**（只能持有者自愿释放）④**循环等待**（T1 等 T2 … Tn 等 T1）。**RAG（资源分配图）**=用有向图表示线程与资源之间请求/分配关系。模型：线程 T1..Tn，资源类型 R1..Rm（CPU 周期、内存、I/O 设备），Ri 有 Wi 个实例；使用流程 Request()→Use()→Release()。请求边 $T_i \rightarrow R_j$，分配边 $R_j \rightarrow T_i$；资源点内画实例数。**无环⇒无死锁**；**有环且每类单实例⇒死锁**；**多实例有环未必死锁**（环外线程完成后释放实例可解环）。**Q5.1–5.3**：P1→R2→P2→R3→P3→R1→P1 成环、全单实例⇒**死锁**。解除：终止一个进程（如 kill P3 释放 R3→P2 完成→P1 完成）或抢占某资源并回滚。**死锁检测算法**（多实例；检查各线程能否最终自行完成）：

```
// Avail=各类空闲实例数, Alloc=某线程已持有, Request=当前请求
[Avail]=[Free]; UNFINISHED=all
do{ done=true
  for node in UNFINISHED:
    if [Request_node] <= [Avail]:
      // Banker 改用 [Max]-[Alloc] <= [Avail]
      remove node; [Avail]+=[Alloc_node]; done=false
}until(done)  // 有剩余⇒死锁; Banker: 全移除⇒safe
```

Q5.4：A=2、B=1；Alloc P1(1,0) P2(1,0) P3(0,1)→Avail=(0,0)；Request P1(0,1) P2(0,1) P3(1,0)→无人满足→UNFINISHED={P1,P2,P3}⇒**死锁**。**Q5.5**：kill P3（释放 B）→P1 得 B 完成释放 (1,1)→P2 完成；或抢占 P3 的 B 并回滚。（若题目给未死锁状态：构造一个使环闭合且无空闲实例的新请求。）**处理死锁四法**：①**预防**=把代码写成不易死锁的形式②**恢复**=让死锁发生，再设法恢复③**避免**=动态延迟资源请求使死锁不发生（Banker）④**否认/鸵鸟算法**=忽略死锁的可能（现代 OS：确保系统自身无死锁，忽略应用死锁）。**预防手段（对应打破条件）**：无限资源/VM 幻觉（打破互斥/等待；“12000 车道的桥”）；不共享（不现实）；**不允许等待**（电话网忙音、以太网冲突退避重试；低效，可能活锁=线程都在动却都无进展）；**一次申请全部**（`Acquire_both(x,y)` 或外加全局锁 z；难预测、易高估；港珠澳大桥一次只一车）→打破持有并等待；**全局顺序加锁**（先 x 后 y；先磁盘后内存；高速公路一律顺时针；网格 **wormhole** 路由（消息像虫子一样占着一串链路前进）用维序：先东西再南北）→打破循环等待。**释放顺序不影响死锁**。**恢复**：终止线程（Godzilla 把车扔河里；但杀持互斥锁的线程会留下不一致状态）；抢占资源（不总符合语义）；**回滚**（把线程恢复到先前检查点重新执行；数据库事务常用；同样方式重启可能再死锁）。VM 换页也可视为抢占内存。**避免**：**Safe**=存在某顺序让所有线程完成（系统可延迟分配防死锁）；**Unsafe**=尚未死锁但不可避免（若线程都申请到最大需求）；Deadlocked⊂Unsafe。避免=不让系统进入 unsafe：请求时 OS 检查，安全则立即给，否则等待。（例：A 拿 x 后，B 请求 y 时应等待——等 A 释放后再给，因为 A 已拿 x 才去拿 y 时检查已太迟。）**Banker（银行家算法）**=动态分配资源，仅当批准后仍存在让所有线程完成的顺序（安全）才批准请求：预先声明 max（最大需求）；每次请求“假装批准”后用 `Max−Alloc≤Avail` 跑检测，安全才批准（保守）。**Banker 通用算例**：总 (10,5,7)。Alloc/Max：P0 (0,1,0)/(7,5,3)、P1 (2,0,0)/(3,2,2)、P2 (3,0,2)/(9,0,2)、P3 (2,1,1)/(2,2,2)、P4 (0,0,2)/(4,3,3)；Avail=(3,3,2)；**Need**（还可能需要的量）=Max−Alloc：P0 (7,4,3)、P1 (1,2,2)、P2 (6,0,0)、P3 (0,1,1)、P4 (4,3,1)。扫描：P1→(5,3,2)、P3→(7,4,3)、P4→(7,4,5)、P0→(7,5,5)、P2→(10,5,7)⇒**safe 序列**（安全序列=按此顺序每个线程都能拿到所需资源并完成）**⟨P1,P3,P4,P0,P2⟩**。P1 请求 (1,0,2)：≤Need 且 ≤Avail，假装后 Avail=(2,3,0)，仍安全⇒**批准**。随后 P4 请求 (3,3,0)>Avail⇒等待；P0 请求 (0,2,0)⇒Avail=(2,1,0)，无人 Need≤Avail⇒**unsafe，拒绝**。

21 · 现代系统：并发与 ML 系统 (L10)

会议（CS 系统领域以会议论文为主要发表形式）：OS 核心 SOSP(1967)、OSDI(1994)、EuroSys(2006)、ATC(1975)；分布式 NSDI/SoCC；网络 SIGCOMM；体系结构 ASPLOS/ISCA/MICRO；PL POPL/PLDI；安全 S&P/CCS/USENIX Sec/NDSS；存储 FAST。**并发 bug**：RaceBench (CCS’23) 可触发、可观测的并发 bug 基准；Go (ASPLOS’19)：阻塞型（线程卡住，如死锁）vs 非阻塞型（如数据竞争），消息传递（channel）并不比共享内存少 bug。**训练**=迭代地找使损失函数最小的权重 W；梯度下降 $W_{k+1} = W_k - \alpha\nabla f(W_k)$；**minibatch**=一次迭代所用的一小批样本；**前向**=逐层计算输出（中间输出称激活）与损失；**反向**=逐层反向传播计算梯度；**worker**=参与训练的一个进程/GPU。**数据并行**=每个 worker 持有完整模型副本、处理不同数据，用 **AllReduce**（所有 worker 求和梯度并把结果发回每个 worker 的集合通信）聚合 $\nabla W = \sum \nabla W_i$→**通信开销高**。**模型并行**=把一份权重按层切分到多个 worker，激活/梯度点对点传→同一时刻只有一个 worker 忙，**硬件利用率低**。另有张量并行（层内切矩阵）、专家并行（MoE 混合专家模型：不同专家子网络放不同 GPU）。**PipeDream (SOSP’19)**：流水线并行=数据并行与模型并行结合、并让多个 minibatch 流水通过各阶段；比 DP（数据并行）快至 **5.3×** 且不损精度。三挑战：①**划分**：各阶段计算 t_i 接近、跨阶段通信最小；**可复制阶段**平衡（如 2-3-2-1）；Profiler（测量每层计算时间与激活大小）+Optimizer（据此求最优划分）依硬件拓扑/模型/内存决定②*1F1B 调度 **（one-forward-one-backward）：稳态每 worker 交替一次前向一次反向，始终忙碌、梯度立即用于更新③*Weight stashing**（权重暂存：保存多个⟨权重,激活⟩版本）：朴素流水中 batch n 前向用 W_n、反向时已是 W_{n+p}→梯度错；存多版本 ⟨权重, 激活⟩，保证同一 batch 前后向同版本；最坏内存≈数据并行。很多模型的中间激活/梯度比 DP 的权重通信小一个数量级。实现与评估：约 3000 行 Python 集成进 PyTorch（DP 基线用 NCCL，PipeDream 用 Gloo，二者都是集合通信库）；3 种集群（4×V100+PCIe+10Gbps、8×V100+NVLink（GPU 间高速互连）+25Gbps、Titan X+40Gbps）；4 类任务（图像分类、翻译、语言模型、视频字幕）；端到端最多快 5.28×（另一例 2.46×）；优化器可能推荐 15-1（第一阶段复制 15 份+第二阶段 1 份）、Straight（纯流水线）或纯数据并行。思考题：PipeDream 如何实现并发并精确控制同步？**Orca (OSDI’22)**：生成式 Transformer 服务（**token**=模型处理/生成的最小文本单元，每次迭代生成一个；**batching 批处理**=把多个请求合并成一次计算）；**迭代级调度**（以模型的一次迭代=生成一个 token 为调度粒度：每次迭代后即可移出完成请求、加入新请求，不必等整批最长者）+ **选择性批处理**（Attention 按请求分别算，线性层把所有 token 拼起来批处理）。**DistServe (OSDI’24)**：**KV cache**=缓存已处理 token 的 Attention 键/值，避免重复计算。Prefill（预填充阶段：一次并行处理整段 prompt 并产生首个 token，**计算密集**，决定 **TTFT**）vs Decoding（解码阶段：逐个生成后续 token，每步读全部 KV cache，**访存密集**，决定 **TPOT**）；**TTFT**=首 token 时延，**TPOT**=每个输出 token 的时延；延迟=TTFT+TPOT×#token。**SLO**（服务等级目标=时延要求）因应用而异（Chat TTFT<1s/TPOT≈100ms；Search TTFT≈200ms；Program TTFT≈200ms/TPOT≈50ms）。目标：最大化 **per-GPU goodput**（满足 SLO 的吞吐）。问题：共置时①prefill-decoding 互相干扰②资源与并行策略耦合（被迫按更苛刻 SLO 超配）。方案：分离到不同 GPU（如 2P1D=2 个 prefill 实例+1 个 decoding 实例），各自选并行策略/资源；挑战：KV cache 传输开销；高节点亲和集群（机间有 InfiniBand 高速网络）分别优化+复制，低亲和集群把同一阶段的 prefill/decoding 放同节点(NVLink)；在线调度减气泡（GPU 空闲等待的时段）、抗突发、周期重规划。效果：**7.4× 请求或 12.6× 更紧 SLO**，>90% 请求满足。评估：4×DGX-A100（每台 8×A100-80GB，机内 600GB/s，跨机 25Gbps），基线 vLLM、DeepSpeed-MII（两种 LLM 推理服务系统），指标 SLO 达成率（虚线=90% 达成点）；KV cache 传输占延迟 <0.1%。**The Bitter Lesson**（Sutton）：人工知识不扩展，学习+搜索随算力扩展；系统释放算力（调度、内存、并行、网络）。AI 需求每 18 个月 10×，远超摩尔定律 2×→专用硬件+集群→OS 要变。

22 · 集群调度论文 (L14)

Firmament (OSDI’16)：集中式调度建模为**最小费用最大流**（在流网络中送出最大流量且总费用最小的优化问题；任务/聚合点（把多台机器归为一类的中间节点）/机器/机架为点，容量=资源上限，费用=策略偏好如数据局部性、负载、等待时间）；集群状态缓变→增量求解，万台机器上亚秒级高质量放置。**Gandiva (OSDI’18)**：内省式（调度器监测作业内部迭代时间与显存行为，而非当黑盒）DL 集群调度：利用 mini-batch 迭代可预测、**迭代边界显存占用低**（激活已释放）→低开销**时间片共享/挂起恢复**、按剖析结果**打包**多作业到一卡、**迁移**做碎片整理与亲和性；改善超参搜索（用不同超参数试跑多个作业、早停差的）等探索作业排队延迟与利用率。**Synergy (OSDI’22)**：打破“每 GPU 按比例绑定 CPU/内存”；在线敏感度剖析各作业对 CPU/内存的吞吐收益，跨作业异构分配，消除数据预处理瓶颈（**data stall**：GPU 因 CPU 喂数据太慢而空转），提升集群吞吐且保持公平。

23 · 易错判断 & 简答模板（Side B 范围）

- SRTF 最优（平均响应时间）但会饥饿且需预知；RR 公平但等长作业时最差；FCFS 等长作业时最优。
- MLFQ 近似 SRTF；Lottery 无饥饿；EEVDF lag O(1) 与 n 无关；WFQ 用虚拟时间因为“顺序不变”。
- 实时≠快；EDF 动态优先级，U≤1 充要（D=P 时）；RMS 静态优先级。
- 单实例有环⟺死锁；多实例有环≠死锁；Unsafe≠Deadlocked；死锁⇒饥饿。
- 打破四条件任一即可预防；**最实用：全局加锁顺序**；释放顺序无关。
- 优先级反转≠死锁（中优先级完成后可恢复），用优先级继承解决。
- Max-min：先满足小需求，再均分；加权解 $\sum \min(r_i, fw_i) = C$。
- 计算题写清：约定（同时到达谁先入队、被抢占者位置）、Gantt、逐项公式、平均值。

24 · 新题型速查：调度 / 公平 / 实时 计算模板

优先级（数值小=高）非抢占，均 0 到：P1(10,pri3) P2(1,1) P3(2,4) P4(1,5) P5(5,2)：P2 0-1, P5 1-6, P1 6-16, P3 16-18, P4 18-19；等待 6,0,16,18,1→**8.2**。**优先级抢占+到达**：P1(到 0,7,pri3) P2(2,4,1) P3(4,1,4) P4(5,4,2)：P1 0-2 | P2 2-6 | P4 6-10 | P1 10-15 | P3 15-16；等待 P1=8, P2=0, P3=11, P4=1→**5.0**。（低优先级 P3 被饿到最后=starvation；可用 aging 提升久等者优先级。）**响应时间（首次运行−到达）别忘**：上面 P1–P4 到达例：RR q=4 响应 0,3,6,9→**4.5**；SRTF 响应 0,0,15,2→**4.25**；FCFS 响应=等待（非抢占）。吞吐=4 个作业/26 单位≈0.154。**实时算法定义**：EDF=绝对截止期最近者优先（动态）；LLF=松弛度 laxity=截止期−当前时间−剩余计算量最小者优先（动态）；RMS=周期越短优先级越高（静态）；DM=相对截止期越短优先级越高（静态）。**嵌套优先级捐赠**：H 等 M 持有的锁，M 又等 L 持有的锁→H 的优先级沿链传给 M 和 L；释放后各自恢复原优先级。**ML 并行类型一句话**：数据并行=复制模型分数据；模型并行=按层切；张量并行=层内切矩阵；专家并行=MoE 不同专家放不同 GPU；流水线并行=按层切+多个 batch 流水。**RR 含切换开销 s**：CPU 有效利用率 $= \frac{q}{q+s}$（例 q=10ms, s=1ms→91%）；n 个进程时最坏等待 $(n-1)(q+s)$；q 远大于 s 才划算。**MLFQ 追踪技巧**：每步记 (队列, 本级已用配额, 剩余总量)；事件优先级：到达/I/O 返回入队 → 抢占检查 → 用满配额降级；同一时刻多个事件时写明约定。若题目有 boost：每 S 时间把所有作业移回 Q0。**Lottery 计算**：A 100 票、B 50、C 250→A 每次中签概率 100/400=25%；n 个时间片中 A 期望 n/4 次；A 首次中签期望等待 1/p=4 个时间片（几何分布）；标准差 ∝ √n→长期才逼近比例（故 lag 为 O(√n)q）。**WFQ 小例**：C=1，两流 w=1，均于 t=0 积压；流 1 包长 2，流 2 两个包长 1：$F_1^1 = 0 + 2 = 2$，$F_2^1 = 0 + 1 = 1$，$F_2^2 = \max(V, F_2^1) + 1 = 2$（积压时 V≤上一个 F，故直接累加）→发送顺序 P_2^1，然后 P_1^1/P_2^2（F 同为 2，按约定打破平局）。权重翻倍 ⇒ 每包虚拟长度减半。**EDF 追踪**：(C,P)=(1,4),(2,6),(3,8)，截止期=下一周期开始；[0,24) 逐单位：1 2 2 3 1 3 3 2 1 2 3 3 1 3 2 2 1 3 2 2 1 3 3 –（平局按任务号），无错过，空闲 1/24=1−U。若 U>1 必有任务错过截止期；RR 在同样任务集下可能错过。**Max-min 加权注水步骤**：①算 $\frac{r_i}{w_i}$ 并升序 ②设 f 使剩余容量在未满足者间按权重分：若最小 $\frac{r_i}{w_i} \leq \frac{C_{\text{rem}}}{\sum w}$ 则满足之，扣除后重算 ③剩下的都得 fw_i。

25 · 新题型速查：同步 / 死锁 编程模板

哲学家就餐（Dining）三解：①资源排序：每人先拿编号小的筷子（最后一人反向）→无环 ②最多 **4** 人入座：`sem room=4` 先 `P(room)` 再拿两根 ③管程一次拿两根：

```
enum {THINK, HUNGRY, EAT} st[5]; cond self[5]; lock L;
pickup(i){ acquire(&L); st[i]=HUNGRY; test(i);
  while (st[i]!=EAT) cond_wait(&self[i], &L);
  release(&L); }
putdown(i){ acquire(&L); st[i]=THINK;
  test((i+4)%5); test((i+1)%5); release(&L); }
test(i){ if (st[i]==HUNGRY && st[(i+4)%5]!=EAT
            && st[(i+1)%5]!=EAT) {
          st[i]=EAT; cond_signal(&self[i]); } }
```

单车道桥 / 同向通行管程（同一方向可多车，异向互斥）：

```
int cnt=0, dir=-1;  lock L; cond ok;
enter(d){ acquire(&L);
  while (cnt>0 && dir!=d) cond_wait(&ok,&L);
  dir=d; cnt++; release(&L); }
leave(){ acquire(&L);
  if (--cnt==0) cond_broadcast(&ok); release(&L); }
// 可能饿死另一方向 → 加计数/轮换
```

用 CV 实现 join：`done=0`；子：`lock; done=1; signal(&cv); unlock;`；父：`lock; while(!done) wait(&cv,&L); unlock;`（while+状态变量，不能只靠 signal——CV 无记忆）。**信号量含义**：若实现允许负值，|s| = 等待者数（课件定义不允许负）。P 前检查值、V 后唤醒——两者都原子。**多实例 RAG 有环无死锁例**：R1×2 分给 T2、T3；R2×2 分给 T1、T4；T1 请求 R1、T3 请求 R2（环 T1→R1→T3→R2→T1）。检测：Avail=(0,0)；T2、T4 无请求→完成→Avail=(1,1)→T1 满足→(1,2)→T3 满足⇒**无死锁**。**给锁顺序的死锁题**：画“持有→等待”图；若存在两线程以相反顺序拿两把锁⇒可能死锁；修复=统一顺序/一次拿全/trylock 失败则释放已持有锁后重试（可能活锁，加随机退避=等待随机时长再重试）。**条件变量 vs 信号量对比**：CV 无值、signal 无人等则丢失、必须在锁内用、wait 释放锁；信号量有计数、V 会被“记住”、可在任意处用；Mesa 下 CV 用 while。**POSIX API 签名速查（写代码题）**：

```
pid_t fork(void);  pid_t wait(int *st);
pid_t waitpid(pid_t pid, int *st, 0);
WIFEXITED(st), WEXITSTATUS(st);  int kill(pid, sig);
int execv(const char *path, char *const argv[]);
int execl(path, arg0, ..., (char*)NULL); // p=搜PATH
int pipe(int fd[2]); int dup2(int old, int new);
pthread_cond_wait(pthread_cond_t*, pthread_mutex_t*);
int pthread_cond_signal(pthread_cond_t*); // _broadcast
pthread_cond_t c = PTHREAD_COND_INITIALIZER;
#include <semaphore.h>
sem_t s; sem_init(&s, 0 /*线程间共享*/, init_val);
sem_wait(&s); /* P */   sem_post(&s); /* V */
```

补充简答：

- **为何内核用独立内核栈**：用户栈不可信（可能非法/被篡改/溢出），陷入内核时需可靠栈保存现场；每线程一个内核栈。
- **为何 syscall 用 trap 而非函数调用**：需原子地切内核态并跳到内核规定入口（向量表），用户不能直接跳内核地址。
- **关中断锁为何多核无效**：只屏蔽本核中断，其他核仍可同时进入临界区。
- **自旋 vs 睡眠**：预计等待短于两次上下文切换开销→自旋（多核）；否则睡眠；单核自旋无意义（持锁者无法运行）。
- **wait() vs pthread_join()**：前者等子进程、拿退出状态；后者等同进程内线程、拿 `void*` 返回值。
- **PipeDream 为何需 weight stashing**：流水线中同一 minibatch 前后向间权重已被其他 batch 更新，不存旧版本则梯度用错权重。
- **Orca 迭代级调度好处**：完成的请求立即离开、新请求立即加入，不必等批中最长输出→降低延迟、提高吞吐。
- **DistServe 为何分离**：prefill 计算密集/TTFT，decode 访存密集/TPOT，混跑相互干扰且被迫共享并行策略。
- **Gandiva 为何在迭代边界挂起**：此时激活已释放、显存占用最小，拷贝/切换开销低。

Banker 请求算法步骤：线程 i 请求 Req：①Req≤Need_i，否则出错（超出声明）②Req≤Avail，否则等待③假装分配：Avail−=Req、Alloc_i+=Req、Need_i−=Req④跑安全性算法：safe→批准；unsafe→回滚并让 i 等待。**RAG 有环且死锁例（多实例）**：R1×1、R2×2、R3×1、R4×3；T1 持 R2 请求 R1；T2 持 R1、R2 请求 R3；T3 持 R3 请求 R2（R2 两个实例都被占）→ 两个环 T1→R1→T2→R3→T3→R2→T1、T2→R3→T3→R2→T2，无人可完成⇒死锁。**排队直觉（L13 knee）**：利用率 U=λ·S（到达率×服务时间）；**M/M/1**（泊松到达、指数服务时间、单服务器的排队模型）响应时间 R=S/(1−U)：U=50%→2S，90%→10S，99%→100S，U→1 时 R→∞。**多级队列 vs MLFQ**：Multilevel queue=作业固定在某队列（如前台/后台）；MLFQ=根据行为在队列间移动（feedback）。**Linux CFS（补充）**：按权重累计 **vruntime**（按权重折算的虚拟运行时间，权重越大增长越慢），红黑树选最小 vruntime，O(log n)，≈加权公平共享；Linux 6.6 起由 EEVDF 取代。**PipeDream 补充**：吞吐由最慢阶段决定：阶段计算时间 2、复制 2 份 → 吞吐 (1/2)×2=1，与计算时间 1 的阶段匹配；对某些算子 ∑W_i 很小，数据并行通信量也小→适合复制。（通用流水线气泡比例≈(p−1)/(m+p−1)，p 阶段 m 个 **micro-batch**（把 minibatch 再切成的更小批），1F1B 稳态无气泡。）**L14 补充**：Firmament：任务→(未调度聚合点|机器/机架)→汇点的流网络，未调度边费用随等待时间增加，避免饿死；放置质量与延迟兼得。Gandiva：打包条件=显存总和≤GPU 容量且合跑吞吐高于分时；还能 grow-shrink（空闲时扩大作业 GPU 数）。Synergy 例：8 GPU/64 核默认每 GPU 8 核；图像模型需 24 核才喂饱 GPU，只给 8 核→GPU 利用率约 33%，同时 3 个语言模型每个只需 2 核→闲置 18 核；按敏感度重新分配即可提升吞吐。**易混名词**：Starvation（饥饿，可能结束）/ Deadlock（循环等待）/ Livelock（都在动但无进展，如反复退避）/ Priority inversion（高等低、中插队）/ Convoy effect（短跟长后）/ Busy waiting（自旋耗 CPU）/ Lost wakeup（检查与睡眠间被唤醒）。

Side A 补遗 · Thread pool / Echo / pipe / ATM 原代码 (L6–L7)

Thread pool（L6）：

```
master() { allocThreads(worker, queue);
  while (1) { conn = AcceptCon();
              Enqueue(queue, conn); wakeup(queue); } }
worker(queue) { while (1) { conn = Dequeue(queue);
  if (conn == NULL) sleepOn(queue);
  else ServeWebPage(conn); } }
```

```
// echo client                     // echo server
fgets(sbuf, MAXIN, stdin);         while (1) {
write(s,sbuf,strlen(sbuf)+1);        n = read(cs,req,MAX);
n = read(s, rbuf, MAXOUT);           if (n <= 0) return;
write(STDOUT_FILENO, rbuf, n);       write(cs, req, n); }
// pipe 父→子（单进程也可自写自读，多线程有用）
pipe(fd);
if (fork()) { close(fd[0]); write(fd[1], msg, len); }
else        { close(fd[1]); read(fd[0], buf, BUFSZ); }
// ATM: 整个 deposit 是临界区
deposit(id, amt) { acquire(&L); acct = get_account(id);
  acct->balance += amt; store_account(acct);
  release(&L); }
// 事件驱动版（单 CPU 重叠 I/O，但须拆成非阻塞片段）
while (1) { ev = wait_for_next_event();
  if (ev == ATMRequest) start_on_request();
  else if (ev == AcctAvail) continue_request();
  else if (ev == AcctStored) finish_request(); }
```

线程版：每请求一线程，按阻塞方式顺序写，更自然。**内核如何服务 read**：库函数把 syscall 号放 %eax、参数放 %ebx…→ trap → 内核取参数、按号分派、读文件、返回值放 %eax → 库取返回值返回。`fread` 多一步：先查用户缓冲区，用完才这样调 `read`，多余数据存入缓冲区。

26 · 课件中抛出的问题 → 参考答案（逐讲）

- **L1** `./cpu A & ./cpu B & ./cpu C` 输出？→ A/B/C 交错且顺序不定（分时）；旧 MacOS9/Win3.1 只会一直打印 A。`memory.c` 两个实例？→ 同一虚拟地址、各自计数（b）。
- **L2** 代码段/数据段/栈/堆里放什么、怎么分配？→ 代码=指令；static data=全局/静态变量；栈=调用时自动分配（局部变量/返回地址），大小有限；堆=malloc 动态分配，程序员负责 free。B&B 下程序能碰到 OS 或其他程序吗？→ 不能（越界会触发异常）。中断目标地址怎么得到？→ 中断向量表。
- **L3** 两个线程栈怎么摆放、最大多大、越界怎么办、怎么发现？→ 地址空间里分开放、设固定上限，越界会覆盖别的栈/堆，可用 guard page 触发异常来发现。内核栈从哪来、是否浪费内存？→ 每线程一个内核栈，代价可以接受。
- **L4** main 不调 exit 就返回？→ 库代为调 exit。fork race 打印什么？→ 父子输出交错、顺序不定；加 sleep 只改变概率，不保证顺序。SIGINT 未注册 handler？→ 进程死（默认动作）。为何 SIGKILL/SIGSTOP 不能改？→ 保证总能终止/暂停失控进程。两任务并发用线程还是进程？→ 见 §5 权衡。
- **L5** lowio.c 读多少字节？→ min(1000, 文件大小)。为何给用户 int 而不是内核指针？→ 内核可校验、用户无法伪造、隐藏内核结构。用户态缓冲 pros/cons？→ §6 表格。
- **L6** 为何共享（别名）打开文件描述是好主意？→ 父子共享资源（同一终端、同一 offset）。socket 怎么 open、命名空间是什么、两端如何在时间上连接？→ socket/bind/listen/accept/connect；IP+端口；server 先 listen，client 后 connect。pipe vs socket vs 文件？→ §7。
- **L7** 两线程各 +1000 次结果？→ 不确定（1899、1937…）。“A wins/B wins” 谁赢？→ 都可能，或永不结束。ATM 怎么修？→ 给 deposit 整段加锁。
- **L8** 能否靠硬件锁指令？→ 硬件更复杂更慢（Intel 432）。sleep 时何时开中断？→ 由下一个线程开。没有其他线程时？→ idle thread。缓冲区第一版能跳出等待循环吗？→ 不能（持锁等待，对方拿不到锁）。第二版一方等另一方时？→ 多核：忙等浪费；单核：只有切换后对方才能进展，更浪费。为何不对称？→ 生产者等空位、发满位；消费者相反。先 P(mutex) 再 P(empty) 行吗？→ 死锁。
- **L9** 为何 while 不用 if？→ Mesa 语义。while 循环是忙等吗？→ 不是，wait 会睡眠。读者会饿死吗？→ 会。删检查/改 broadcast/只用一个 CV？→ §11。
- **L11** 时间片怎么选？→ 远大于切换开销，10–100ms。RR 总比 FCFS 好吗？→ 否（等长作业时更差）。怎么预测未来？→ 用户申报或指数平均。burst 短就是交互式吗？→ 常用启发式，但可被利用。
- **L12** 短作业太多时 lottery 怎么办？→ 负载太高大家都慢（注销用户）。为何用虚拟时间？→ 完成顺序不随新到达改变。EEVDF 解决什么问题？→ WFQ 的 O(n) lag。
- **L13** 自旋锁何时更好？→ 多核上短等待，比如 barrier。调度器怎么知道该捐赠优先级？→ 锁记录持有者，阻塞时把优先级借给持有者。释放锁的顺序重要吗？→ 不重要。k 只手的哲学家？→ 最后一根、且没人能凑齐 k 根时不许拿；倒数第二根、且没人能凑齐 k−1 根时不许拿……以此类推。何时买更快硬件？→ 负载曲线到拐点时。

✍ 手写补充区（考前补充个人易错点 / 老师复习课强调内容）